[GHC] Template Haskell: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/template_haskell.html
[Haskell Wiki] template haskell: https://wiki.haskell.org/Template_Haskell
[Hackage] template-haskell: https://hackage.haskell.org/package/template-haskell
Template meta programming for Haskell (origin paper): https://www.microsoft.com/en-us/research/publication/template-meta-programming-for-haskell

```
-- Language.Haskell.TH
data Q
instance MonadFail Q, MonadFix Q,
  Applicative Q, Functor Q, Monad Q,
  Quote Q, Quasi Q,
  Monoid a => Monoid (Q a), SemiGroup a => SemiGroup (Q
a)

class Monad m => Quote m where
  newName :: String -> m Name

runQ :: Quasi m => Q a -> m a
runIO :: IO a -> Q a
reportError :: String -> Q ()
reportWarning :: String -> Q ()
fail :: String -> Q a
recover :: Q a -> Q a -> Q a
location :: Q Loc

data Loc = Loc { loc_filename :: String, loc_package ::
String, loc_module :: String, loc_start :: String,
loc_end :: String }


-- name
data Name
data NameSpace
mkName :: String -> Name
nameBase :: Name -> String
nameModule :: Name -> Maybe String
namePackage :: Name -> Maybe String
nameSpace :: Name -> Maybe String
tupleTypeName :: Int -> Name
tupleDataName :: Int -> Name
...
```

Template Haskell
A type of **meta programming.**
Generate code **programatically**. That is, write code to generate **AST** directly, which is converted into code at compile time.

splice $	AST -> code

splice = **inject** code, from Template meta programming for Haskell: https://www.microsoft.com/en-us/research/

publication/template-meta-programming-for-haskell

quasiquote [| ... |] code -> AST
[| ... |] = **Oxford Bracket Syntax**
[| ... |] = [e| ... |] = expression quote
e.g. [| 1 |] :: Q Exp = LitE (IntegerL 1)
[t| ... |] = type quote
e.g. [t| Int |] :: Q Type = AppT ListT (ConT Int)
[p| ... |] = pattern quote
e.g. [p| (a,b) |] :: Q Pat = TupP [VarP (mkName "a"),VarP (mkName "b")]
[d| ... |] = declaration quote
e.g. [d| x = 1 |] :: Q Dec = [ValD (VarP (mkName "x")) (NormalB (LitE (IntegerL 1))) []]

custom quote [someQuote| ... |] String -> AST

e.g.

```
-- quote
[| 1 |] :: Q Exp              -- AST from literal
pure $ LitE (IntegerL 1)      -- same
litE (IntegerL 1)             -- same

-- splice
$[| 1 |] :: Int               -- splice i.e. execute AST
$[| 1 |] == 1                 -- True
$(litE (IntegerL 1)) == 1     -- same with raw AST

-- nesting
$[| $[| 1 |] |] * 2                    -- 2
$[| $[| 1 |] * 2 |]                    -- same
$[| $(litE (IntegerL 1)) * 2 |]        -- same with raw
AST
$[| $(litE (IntegerL 1)) * $[| 2 |] |]
```

e.g. use in GHCI

```
ghci> :set -XTemplateHaskell
ghci> import Language.Haskell.TH

ghci> [| 1 |]
     -- AST of 1
LitE (IntegerL 1)
ghci> :t [| 1 |]
[|1|] :: forall {m :: Type -> Type}. Quote m => m Exp
ghci> litE (IntegerL 1))
     -- raw AST
LitE (IntegerL 1)

ghci> $[| 1 |]
1
ghci> $[| 1 |] == 1
True
ghci> $[| $[| 1 |] |] == 1
True
ghci> $(litE (IntegerL 1)) == 1
```

```
True
```

AST
Language.Haskell.TH.Syntax: https://hackage.haskell.org/package/template-haskell-2.19.0.0/docs/Language-Haskell-TH-Syntax.html

Lit (Literal)

```
-- Language.Haskell.TH.Syntax
data Lit where
 CharL Char                    literal 'a'
 | StingL String               "hello"
 | IntegerL Integer            1
 | Rational Rational           1.3
 | IntPrimL Integer            1# i.e. primitive integer
 | WordPrimL Integer
 | ...
```

Type

```
-- Language.Haskell.TH.Syntax
data Type =
 ForAllT [TyVarBndr Specificity] Cxt Type       forall
<vars>. <ctxt> => <type>
 | ForAllVisT [TyVarBndr ()] Type               forall
<vars> -> <type>
 | AppT Type Type                               T a b
 | AppKindT Type Kind                           T @k t
 | SigT Type Kind                               t :: k
 | VarT Name                                    a
 | ConT Name                                    T
 | TupleT Int                                   (,) (,,)
etc
 | ListT                                        []
 | ArrowT                                       (->)
 | LitT TyLit                                   0,
1, "a"
 ...


type Cxt = [Pred]            (Read a, Show a) => ...
type Pred = Type

data TyVarBndr flag =
  PlainTV Name flag                 a
  | KindedTV Name flag Kind         (a :: k)

data Specificity =
  SpecifiedSpec                     a
  | InferredSpec                    {a} in forall {a}.

data TyLit =
```

```
  NumTyLit Int                          1 :: Nat
  | StrTyLit String                     "abc" :: Symbol
  | CharTyLit Char
```

Dec (declarations)

```
-- Language.Haskell.TH.Syntax
data type Dec =
   FunD Name [Clause]                                 f x y
= x + y where ...
 | ValD [Pat] Body [Dec]                              v = x
where ...
 | DataD Ctx Name [TyVarBndr ()]                      data
<ctx> => T a b = A a | B b deriving (Show, Eq)
    (Maybe Kind) [Con] [DeriveClause]
 | NewTypeD Ctx Name [TyVarBndr ()]
newtype <ctx> => T a = T a deriving (Show, Eq)
    (Maybe Kind) Con [DeriveClause]
 | TySynD Name [TyVarBndr ()] Type                    type
A a = [a]
 | ClassD Ctx Name [TyVarBndr ()]                     class
<ctx> => MyClass a where ...
    [FunDep] [Dec]
 | InstanceD (Maybe Overlap) Ctx Type [Dec]
instance {-# Overlaps #-} Show a => Show (A a)
where ...
 | SigD Name Type                                     f ::
<type>
 | KiSigD Name Kind                                   type
A :: Type -> Type
 | ForeignD Foreign
 | InfixD Fixity Name
infixl 6 +
 | DefaultD [Type]
 | PragmaD Pragma                                     {-#
LANGUAGE TemplateHaskell #-}
 | DataFamilyD Name [TypVarBndr ()] (Maybe Kind)      data
family T a b :: Type
 | DataInstD ...
 | NewTypeInstD ...
 | TySyncInstD TypSynEq
 | OpenTypeFamilyD TypeFamilyHead                     type
family A a b = (r :: Type) | r -> a b
 | ClosedTypeFamilyD TypeFamilyHead [TySynEqn]        type
family A a b = (r :: Type) | r -> a b where ...
 | RoleAnnotD Name [Role]
 | StandaloneDerivD (Maybe DeriveStrategy)
deriving stock instance Show a => Show (A a)
    Ctx Type
 | PatSynD ...
 | PatSynSigD ...
 | ImplicitParamBind ....
```

```
data Clause = Clause [Pat] Body [Dec]              a b =
<body> where <declarations>
data Con =
   NormalC Name [BangType]                         A a b
 | RecC Name [VarBangType]                         A
{ getA :: a, getB :: b }
 | InfixC BangType Name BangType                   a :+
b
 | ForallA [TypVarBndr Specificity]                A ::
forall a b. (Eq a, Eq b) => a -> b -> A a b
    Ctx Con
 | GadtC [Name] [BangType] Type                    A ::
a -> b -> A a b
 | RecGadtC [Name] [VarBangType] Type              A ::
{ getA :: a } -> A a b

type BangType = (Bang, Type)
type VarBangType = (Name, Bang, Type)
```

Expr (expressions)

```
data Exp
   VarE Name                                 x
 | LitE Lit                                  1 i.e. a
liteal
 | ConE Name                                 Just
 | AppE Exp Exp                              f x
 | AppTypeE Exp Type                         f @Int
 | InfixE (Maybe Exp) Name (Maybe Exp)       x+y  or  (x+)
 or (+y)
 | UInfixE Exp Name Exp
 | ParensE Exp                               (<exp>)
 | LamE [Pat] Exp                            \a b -> <exp>
 | LamCaseE [Match]                          \case -> ...
 | LamCasesE [Clause]
 | TupE [Maybe Exp]                          (a,b) or (a,)
 | CondE Exp Exp Exp                         if b then x
else y
 | MultiIfE [(Guard, Exp)]
 | LetE [Dec] Exp                            let x = 1 in x
 | CaseE Exp [Match]                         case x of ...
 | DoE (Maybe ModName) [Stmt]                do { ... }
 | MDoE (Maybe ModName) [Stmt]               mdo { ... }
 | CompE [Stmt]                              [ x | x <- y]
 | ArithSeqE Range                           [1..10]
 | ListE [Exp]                               [1,2,3]
 | SigE Exp Type                             (1 :: Int)
 | RecConE Name [FieldExp]                   MyValue
{ x=1,y=2 }
 | RecUpE Exp [FieldExp]                     myvalue
{ x=1 }
 | ...
...
```

```
data Match
 Match Pat Body [Dec]          case e of { (x,y) <-
<body1> where <dec1> ... }
```

Pat (Pattern)

```
data Pat
 LitP Lit                       1
 | VarP Name                    x
 | TupP [Pat]                   (x,y)
 | ConP Name [Type] [Pat]       (MyType @Int @String x
y)
 | InfixP Pat Name Pat          x : y
 | UInfixP  Pat Name Pat
 | ParensP Pat                  (<exp>)
 | TildeP Pat                   ~x
 | BangP Pat                    !x
 | AsP Name Pat                 @v(x,y)
 | WildP                        _
 | RecP Name [FieldPat]
 | ListP [Pat]                  [x,y,z]
 | SigP Pat Type                x :: Int
 | ViewP Exp Pat                <exp> -> x
```

Body

```
data Body
 NormalBody Expr                        f x = <expr>
 | GuardedBody [(Guard, Expr)]          f | x1 <- x =
<expr> | x2 <- x = <expr> ...
```

Quotes
Language.Haskell.TH.Lib: https://hackage.haskell.org/package/template-haskell-2.19.0.0/docs/Language-Haskell-TH-Lib.html

```
-- Language.Haskell.TH.Lib
forallT :: Quote m => [TyVarBndr Specificity] -> m Cxt
-> m Type -> m Type
appT :: Quote m => m Type -> m Type -> m Type
appKindT :: Quote m => m Type -> m Kind -> m Type
varT :: Quote m => Name -> m Type
conT :: Quote m => Name -> m Type
...

valD :: Quote m => m Pat -> m Body -> [m Dec] -> m Dec
funD :: Quote m => Name -> [m Clause] -> m Dec
sigD :: Quote m => Name -> m Type -> m Dec
```

Basics

```
[| ... |] :: Quote m => m Exp
[e| ... |] :: Quote m => m Exp
[t| ... |] :: Quote m => m Type
[d| ... |] :: Quote m => m Dec
[p| ... |] :: Quote m => m Pat
```

```
-- quasi quote [| ... |]: code -> AST
--
{-# LANGUAGE TemplateHaskell #-}
[| 1 |] :: Quote m => m Exp                -- Expression
AST
[e| 1 |]                                   -- same
litE (IntegerL 1)  :: Quote m => m Exp     -- same
handwritten AST

[t| Int |] ::Quote m => m Type             -- type
[d| data A = A |] :: Quote m => m Dec      --
declaration
[p| Just _ |] :: Quote m => m Pat          -- pattern
```

```
-- e.g. basic usage

{-# LANGUAGE TemplateHaskell #-}

-- quote types
$[d|                              -- declaration
   myValue :: Int
   myValue = 1
   |]
myId :: $[t|Int -> Int|]        -- type
myId $[p|x|] = $[|x|]           -- pattern, expression


-- result
myValue         -- 1
myId 10         -- 10
```

```
-- splice: AST -> code
--
{-# LANGUAGE TemplateHaskell #-}
$[| 1 |] :: Int                            -- splice
 i.e. execute AST
$(litE (IntegerL 1))                       -- same
x = $[| 1 |]                               -- e.g. x =1
x = $(litE (IntegerL 1))


-- nesting i.e. mixing splice + quote
```

```
--
$[| $[| 1 |] |] * 2                          -- 2
$[| $[| 1 |] * 2 |]                          -- same
$[| $(litE (IntegerL 1)) * 2 |]              -- same with
raw AST
$[| $(litE (IntegerL 1)) * $[| 2 |] |]


-- quote scope: i.e. names captured in quote
--
-- Any name outside of quote
x = 1 :: Int
$[| x |]                                -- 1. Capture name
in code.
$[| let y = 2 in $[| y |]|]             -- 2. Capture name
in outer quote.


-- names
--
mkName :: String -> Name
mkName "x"
'x                          -- same as mkName "x", if it
exists
'abcd                       -- ERROR if it does not exist
mkName "Int"                -- for types
''Int                       -- same
''BADTYPE                   -- ERROR if it does not exist
```

```
x = 1
y :: Int; y = x
$[d| y :: $(conT (mkName "Int")); y = $(varE
(mkName "x")) |]    -- same
$[d| y :: $(conT ''Int); y = $(varE 'x) |]
        -- same
$[d| y :: $(conT ''Int); y = $(varE 'xxx) |]
         -- ERROR if xxx does not exist
```

Example: pick ith value of any tuple
Generalize fst, snd to any tuple
i.e.

$(ithOfTuple 2 0) :: (a, b) -> a	same as fst
$(ithOfTuple 3 0) :: (a,b,c) -> a	fst for triple
$(ithOfTuple 3 1) :: (a,b,c) -> b	snd for triple
$(ithOfTuple 3 2) :: (a,b,c) -> c	
$(ithOfTuple 3 3)	ERROR

```
module Tuple (ithOfTuple) where

import Control.Monad
import Language.Haskell.TH

ithOfTuple :: Int -> Int -> Q Exp
```

```
ithOfTuple n i = do
  when (n <= i) $ fail $ "Error: " <> show n <> " <= "
<> show i
  name <- newName "x"
  let pickNth :: [Q Pat]
      pickNth = (\j -> if j == i then varP name else
wildP) <$> [0 .. (n - 1)]
  lamE
    [tupP pickNth]
    (varE name)

$(ithOfTuple 2 0) (True, 1)             -- ERROR. Cannot
use in same module
```

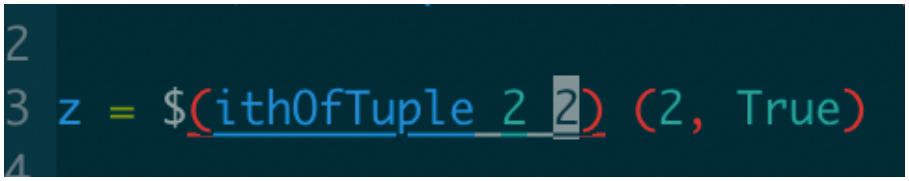
Use

```
import Tuple

$(ithOfTuple 2 0) (True, 1)             -- True
$(ithOfTuple 2 1) (True, 1)             -- 1
$(ithOfTuple 3 0) (True, 1, 'a')        -- Tuple of
different arity
$(ithOfTuple 3 1) (True, 1, 'a')
$(ithOfTuple 3 2) (True, 1, 'a')

$(ithOfTuple 2 2) (True, 1)             -- "Error: 2 <=
3"
$(ithOfTuple 2 3) (True, 1)             -- "
```

Error



Example: printf
Example from "Template Meta-Programming for Haskell": https://www.microsoft.com/en-us/research/publication/template-meta-programming-for-haskell
i.e. the printf function in Unix/C

```
$ printf "Hello %s" A                  # "Hello A"
$ printf "Hello %s %d" A 1             # "Hello A 1"
```

2 steps:
1) **parse** the format string for format characters
  Uses **attoparsec** parser to parse format string
  attoparsec: https://hackage.haskell.org/package/attoparsec
2) compute new string
  e.g. "Hello %d %s" --parse--> ["Hello ", D, " ", S] --compute--> "Hello 1 A" D=1 S="A"

PrintfParser
Parse format string

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}

module PrintfParser
  ( parseOnly,
    parseD,
    parseL,
    parseS,
    parseFormats,
    parseLiteralChar,
    isFormatValue,
    Format (..),
  )
where

import Control.Applicative
import Data.Attoparsec.ByteString.Char8 hiding (D)
import Data.Attoparsec.Combinator
import Data.String

data Format where
  Literal :: String -> Format
  D :: Format
  S :: Format
  deriving stock (Show, Eq)

dValue :: IsString s => s
dValue = "%d"

sValue :: IsString s => s
sValue = "%s"

isFormatValue :: (Eq s, IsString s) => s -> Bool
isFormatValue s
  | s == dValue = True
  | s == sValue = True
  | otherwise = False

parseD :: Parser Format
parseD = string dValue >> pure D <?> "D"

parseS :: Parser Format
parseS = string sValue >> pure S <?> "S"

parseL :: Parser Format
parseL = Literal <$> many1 parseLiteralChar <?>
"Literal"

-- note: parse exactly one character that isn't a format
character
parseLiteralChar :: Parser Char
parseLiteralChar =
```

```
  ( do
      v <- lookAhead $ count 1 (parseD <|> parseS) <|>
pure []
      if null v
        then anyChar
        else fail ("Encountered format " <> show (head
v))
  )
    <?> "literal char"

parseFormats :: Parser [Format]
parseFormats = many1 (parseD <|> parseS <|> parseL)
```

Printf

```
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TupleSections #-}

module Printf (printf) where

import Data.Attoparsec.ByteString.Char8 hiding (D)
import Data.ByteString.Char8 qualified as B
import Language.Haskell.TH
import PrintfParser

printf :: String -> Q Exp
printf str =
  either
    (\s -> fail $ "Failed to parse string " <> s)
    (flip compute [|""|])
    v
  where
    v = parseOnly parseFormats (B.pack str)

compute :: [Format] -> Q Exp -> Q Exp
compute [] resultText = resultText
compute ((Literal l) : formats) resultText = compute
formats [|$resultText <> l|]
compute (D : formats) resultText = [|\(v :: Int) -> $
(compute formats [|$resultText <> show v|])|]
compute (S : formats) resultText = [|\v -> $(compute
formats [|$resultText <> v|])|]
```

e.g.

```
$(printf "Hello")                          -- "Hello"
$(printf "Hello %s") "ABC"                 -- "Hello ABC"
$(printf "Hello %s %d") "ABC" 1            -- "Hello ABC
1"
```

[Troubleshooting] error "Parse error: module header, import declaration or

top-level declaration expected." when loading declaration

e.g. Define declaration

```
{-# LANGUAGE TemplateHaskell #-}

module MyModule where
declareX = [d| x = 1 |] :: Q [Dec]
```

BAD: Error when executing declaration

```
# Another module
import MyModule
declareX                -- ERROR "Parse error: module
header, import declaration or top-level declaration
expected."
```

GOOD: Must have TemplateHaskell

```
{-# LANGUAGE TemplateHaskell #-}

# Another module
import MyModule
declareX                -- good

y = x                   -- x is defined as x=1
```



Cause
Ref: https://stackoverflow.com/questions/50197631/haskell-parse-error-module-header-import-declaration-or-top-level-declaration
This actually has nothing to do with templates.
Just that unless **TemplateHaskell** extension is enabled, it treats the template declaration as an expression

==========

Reification

Language.Haskell.TH Reify: https://hackage.haskell.org/package/template-haskell-2.19.0.0/docs/Language-Haskell-TH.html#g:4

```
reify :: Name -> Q Info
reifyModule :: Module -> Q ModuleInfo
lookupTypeName :: String -> Q (Maybe Name)
lookupValueName :: String -> Q (Maybe Name)
reifyFixity :: Name -> Q (Maybe Fixity)
```

```
reifyType :: Name Q Type
reifyInstances :: Name -> [Type] -> Q [InstanceDec]

data Info
 ClassI Dec [InstanceDec]                              --
 type class
 | ClassOpI Name Type PparentName                      --
 type class method
 | TyConI Dec                                          --
 type constructor
 | FamilyI Dec [InstanceDec]                           --
 type or data family
 | PrimTyConI Name Arity Unlifted                      --
 primitive type constructor e.g. Int#
 | DataConI Name Type ParentName                       --
 data type e.g. Just, Nothing
 | PatSynI Name PatSynType                             --
pattern synonym
 | VarI Name Type (Maybe Dec)                          --
variable
 | TyVarI Name Type                                    --
type variable
```

Reify
Given a **name**, convert to its **AST**.
AST shows what the name represents i.e. whether it is a type name, a data constructor name, a variable, a type variable, etc

Example: get object type of a name
e.g.

Bool	Type constructor name
True	Data constructor name
Num	Class name
...	

```
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE TemplateHaskell #-}

module Reify (getNameInfo) where

import Data.Functor
import Language.Haskell.TH

getNameInfo :: Name -> Q Exp
getNameInfo name = do
  (a, b) <- _getNameInfo name
  [|(a, b)|]

_getNameInfo :: Name -> Q (String, String)
_getNameInfo name =
  reify name <&> \case
    ClassI {} -> ("class", nameBase name)
    ClassOpI {} -> ("class method", nameBase name)
```

```
    TyConI {} -> ("type", nameBase name)
    FamilyI {} -> ("type family", nameBase name)
    PrimTyConI {} -> ("primitive type constructor",
nameBase name)
    DataConI {} -> ("data constructor", nameBase name)
    PatSynI {} -> ("pattern", nameBase name)
    VarI {} -> ("variable", nameBase name)
    TyVarI {} -> ("type variable", nameBase name)
```

e.g.

```
{-# LANGUAGE MagicHash #-}
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}

import GHC.Base
import GHC.Generics
import Reify

typeInfos :: [(String, String)]
typeInfos =
  [
    $(getNameInfo ''Show),                --
("class", "Show")
    $(getNameInfo 'show),                 -- ("class
method", "show")
    $(getNameInfo ''Bool),                -- ("type",
"Bool")
    $(getNameInfo ''GHC.Generics.Rep),    -- ("type
family", "Rep")
    $(getNameInfo ''GHC.Base.Int#),       -- ("primitive
type constructo", "Int#")
    $(getNameInfo 'True),                 -- ("data
constructor", "True")
    $(getNameInfo 'id)                    --
("variable", "id")
  ]
```

Example: auto generate is* functions for data types

e.g.

isTrue :: Bool -> Bool
isFalse :: Bool -> Bool
isJust :: Maybe a -> Bool
isNothing :: Maybe a -> Bool

```
{-# LANGUAGE TemplateHaskell #-}
module Reify (genIs) where

import Language.Haskell.TH

genIs :: Name -> Q [Dec]
genIs typeName = do
```

```
  (mainType, cs) <- reify typeName >>=
getDataConstructorInfo
         -- reify
  concat <$> traverse (genIsForConstructor mainType) cs

getDataConstructorInfo :: Info -> Q (Type, [(Name,
Int)])
getDataConstructorInfo (TyConI (DataD _ typeName
typeVars _ dataCons _)) = do
  dataConInfo <- traverse getDataConInfo dataCons
  return (mainType, dataConInfo)
  where
    mainType = applyTypes (ConT typeName) $ VarT .
mkName . ("a" <>) . show <$> [1 .. length typeVars]
    getDataConInfo (NormalC conc vars) = pure (conc,
length vars)
    getDataConInfo (RecC conc vars) = pure (conc, length
vars)
    getDataConInfo _ = fail ""
getDataConstructorInfo info = fail $ "Not a data type:"
<> show info

genIsForConstructor :: Type -> (Name, Int) -> Q [Dec]
genIsForConstructor mainType (conName, conVars) = do
  let predicateName = mkName $ "is" <> nameBase conName
      conPattern = ConP conName [] (replicate conVars
WildP)
  -- e.g.
  --    isJust :: Maybe a -> Bool
  --    isJust (Just _) = True
  --    isJust _ = False
  signature <-
    sigD
      predicateName
      [t|$(pure mainType) -> Bool|]
  expression <-
    funD
      predicateName
      [ clause [pure conPattern] (normalB [|True|]) [],
        clause [wildP] (normalB [|False|]) []
      ]
  return [signature, expression]
```

usage

```
{-# LANGUAGE TemplateHaskell #-}

import Reify

-- step: generate
$(genIs ''Bool)
$(genIs ''Maybe)
$(genIs ''Either)
$(genIs ''id)                 -- ERROR. "Not a data
```

```
type:VarI GHC.Base.id (ForallT ...)"

-- generated functions
isTrue :: Bool -> Bool
isFalse :: Bool -> Bool
isJust :: Maybe a -> Bool
isNothing :: Maybe a -> Bool
isLeft :: Either a b -> Bool

-- e.g.
isTrue True                    -- True
isTrue False                   -- False
isJust (Just 'a')              -- True
isJuST Nothing                 -- False
isLeft (Left 'a')              -- True
isLeft (Right 'a')             -- False
```

==========

Quasiquoter

[GHC] QuasiQuotes extension: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/template_haskell.html#template-haskell-quasi-quotation
Language.Haskell.TH.Quote: https://hackage.haskell.org/package/template-haskell/docs/Language-Haskell-TH-Quote.html

```
data QuasiQuoter = QuasiQuoter {
  quoteExp :: String -> Q Exp,
  quoteDec :: String -> Q [Dec],
  quoteType :: String -> Q Type,
  quotePat :: String -> Q Pat
}
```

QuasiQuoter
Custom quasiquote

e.g.

```
{-# LANGUAGE TemplateHaskell #-}

import Language.Haskell.TH
import Language.Haskell.TH.Quote

simple :: QuasiQuoter
simple = QuasiQuoter {
  quoteExp = \s -> ...             -- String -> Q Exp
  quoteDec = \s -> ...             -- String -> Q [Dec]
  quoteType = \s -> ...            -- String -> Q Type
```

```
  quotePattern = \s -> ...          -- String -> Q Pat
}

[simple| ... |]                     -- ERROR. Can't use
in same module
```

use in another module

```
{-# LANGUAGE QuasiQuotes #-}      -- IMPORTANT

[simple| ... |]                      -- call quoteDec
$[simple|...|]                       -- ERROR. No need to
splice
x = [simple| ... |]                  -- call quoteExp
f [simple| ... |] = 0                -- call quotePat
f :: [simple| ... |]                 -- call quoteType
```

Example: basic usage

```
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}

module QuasiQuote (simple) where

import Language.Haskell.TH
import Language.Haskell.TH.Quote

simple :: QuasiQuoter
simple =
  QuasiQuoter
    { quoteExp = litE . StringL,
                 -- as String
      quoteDec = \s ->
               -- <name> :: Int; <name> = 1
        ( do
            sig <- sigD (mkName s) [t|Int|]
            value <- funD (mkName s) [clause [] (normalB
[|1|]) []]
            return [sig, value]
        ),
      quotePat = varP . mkName,
                 -- \<name> = ...
      quoteType = conT . mkName
                 -- v :: <name>
    }

[simple| ... |]                     -- ERROR. Can't use
in same module
```

usage

```
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}

import QuasiQuote

x = [simple|hello|]              -- expression
[simple|myValue|]                -- declaration
f :: [simple|Int|] -> Int        -- type
f [simple|x|] = x * 100          -- pattern
$[simple|myValue|]               -- ERROR. No need to
splice

-- e.g.
x                     -- "hello"
myValue               -- 1
f 10                  -- 1000
```

Example: multiline string with quasiquote

```
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}

module MyQuote (simple) where

import Language.Haskell.TH
import Language.Haskell.TH.Quote

simple :: QuasiQuoter
simple =
  QuasiQuoter
    { quoteExp = litE . StringL,
                      -- multiline string
      quoteDec = \s -> fail "Not implemented",
      quoteType =  \s -> fail "Not implemented",
      quotePat =  \s -> fail "Not implemented"
    }
```

usage

```
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}

import MyQuote

-- Expression
txt :: String
txt = [simple|hello            --
multiline "hello\na\nb\nc"
a
b
```

```
c|]
```

==========
haskell-src-meta

haskell-src-meta: https://hackage.haskell.org/package/haskell-src-meta-0.8.12

```
-- Language.Haskell.Meta.Parse
parseExp :: String -> Either String Exp
parseType :: String -> Either String Type
parseDec :: String -> Either String Dec
parsePat :: String -> Either String Pat
```

Meta
Parse text -> AST
Primarily for parsing string in a custom quote

Example: trivial quote i.e. parse in usual way

```
module Template.Parse (trivial) where

import Control.Arrow ((>>>))
import Language.Haskell.Meta.Parse
import Language.Haskell.TH
import Language.Haskell.TH.Quote

trivial :: QuasiQuoter
trivial =
  QuasiQuoter
    { quoteExp = parseWith parseExp,
      quoteDec = parseWith parseDecs,
      quoteType = parseWith parseType,
      quotePat = parseWith parsePat
    }

parseWith :: (String -> Either String a) -> String -> Q
a
parseWith parser = parser
    >>> either
      (fail . ("Failed to parse:" <>))
      pure
```

usage

```
{-# LANGUAGE QuasiQuotes #-}
{-# LANGUAGE TemplateHaskell #-}
```

```
[trivial|                                   -- Dec
myValue :: Int
myValue = 1
 |]

myFn :: [trivial| Int -> Int |]            -- Type
myFn [trivial|x|]                          -- Pat
  = [trivial| x * 10 |]                    -- Exp

-- e.g.
myValue               -- 1
myFn 3                -- 30
```

==========
======